

① 并行計算痛點在於 $\left\{ \begin{array}{l} \text{communication overhead} \\ \text{load imbalance} \end{array} \right.$, 注意 ① 工作資源分配 ② 資料搬移 ③ 重組

② Power wall. $P \propto f^3$ (電壓隨頻率調升)

$\Rightarrow$ 走向 parallelism

③ superscalar 和 ILP 因 Data Hazard 和 Control Hazard, 只能提升 3~4 倍

④ 把 task hardware 切成固定 N threads, 跨硬體無法充分利用資源 (以 forall 和 openMP 指令)

⑤ instruction fetch/decode 會消耗晶片面積, 故改用一個 ALU 控多個 ALU 混用一個指令。
但也因採用同一組 ALU, 故 if-else 會做 hardware mask, 會有多 cores idle.

⑥ 只要 thread cache miss, 等 mem 時 thread 切去做 ready threads.
多 threads 共享 L1 cache 可能有 cache thrashing

在 gcc 的 `__attribute__((aligned(64)))`:
去對齊 mem, 因此 cache / reg 表取才不會
發生 thrashing 或 false sharing

綜合來看: 資料有 locality, 多運算場景

動態分配: 透過 job queue (atomic counter), 動態鎖工作
lock 鎖住資源

fork-join. 隨 Runtime 展升平行

注意 false sharing: 每 thread 各自變數落在同一 byte cache 中, 取用時一直 miss.
叫 cache ping pong

cuda 執行 $\left\{ \begin{array}{l} \text{分 threads 和 blocks} \\ \text{以 32 threads 為 1 warp 做同一指令} \\ \text{if divergence, 部份 threads 只能 idle} \end{array} \right.$ 以 blocking / tiling 做資料分體

在 block 層中的 shared memory, thread 搬 data 便要全体 sync.
因為用 mesh + id 做 index, 故為 lock free